The Chess Algorithm: A Novel Metaheuristic Optimization Technique with Applications in Transportation Network Engineering

Seyed Saber Naseralavi Seyedali Mirjalili

2026-07-04

We introduce the Chess Algorithm (CA), a population-based metaheuristic that borrows its search logic from the strategy of chess rather than from foraging or hunting behavior in nature. The population is not homogeneous: at every iteration agents are ranked and assigned the roles of King, Queens, Rooks, Bishops, Knights, and Pawns, and each role moves according to an operator abstracted from the geometry of the corresponding piece. Strategic ideas from the game supply the control machinery. Development governs the exploration–exploitation schedule; sacrifice permits worse moves under a Metropolis-type rule restricted to minor pieces; pinning progressively freezes coordinates around the incumbent; castling exchanges coordinate blocks between the King and the best Rook when doing so helps; en passant performs an opportunistic, self-adaptive local search; and a threefold-repetition rule restores diversity whenever the population collapses onto the incumbent. We evaluate CA on six standard 30-dimensional benchmarks against GA, PSO, SA, and GWO under matched evaluation budgets (30 runs, ANOVA and Wilcoxon tests), where it significantly outperforms GA, PSO, and SA on most of the suite while GWO keeps its known edge on the classical functions. Practical value is then assessed on transportation problems. On the coordinated signal timing of an eight-intersection arterial with a Webster/HCM delay model, CA matches the best solution any competitor finds. In an extended comparison against six algorithms from the independent mealpy library (WOA, SCA, ALO, MFO, HHO, and DE) on the signal problem and on a continuous berth allocation instance with a known global optimum, CA's placement is assessed against third-party implementations rather than our own. The results support a measured claim: CA is a sound, competitive, and fully reproducible addition to the toolbox of transportation network optimization.

[1] *Seyed Saber Naseralavi (corresponding author,* `saber_naseralavi@uk.ac.ir`*) — Department of Civil Engineering, Faculty of Engineering, Shahid Bahonar University of Kerman, Kerman, Iran.*

[2] *Seyedali Mirjalili — Centre for Artificial Intelligence Research and Optimization, Torrens University Australia, Brisbane, Australia.*

An HTML version of this article, together with all code and data, is available at https://sabernaseralavi-60.github.io/2026_Chess-Algorithm/.

Author note. Dr. Seyedali Mirjalili is listed as an invited co-author; his participation is pending confirmation. The invitation reflects how much this study owes to his work on swarm and nature-inspired optimization, and his name will be retained only with his

explicit consent.

Keywords: metaheuristics; chess algorithm; global optimization; benchmark functions; traffic signal timing; berth allocation; transportation networks.

1 Introduction

1.1 Motivation

A large share of the decisions a transportation engineer has to make can be written, after enough simplification, as a global optimization problem:

$$\min_{\mathbf{x}\in\Omega} f(\mathbf{x}), \qquad \Omega = \{\mathbf{x} \in \mathbb{R}^D : l_j \leq x_j \leq u_j,\ j = 1, \dots, D\}. \tag{1}$$

The trouble is what f looks like in practice. Signal timing, network design, transit scheduling, detector placement — in each of these the objective is non-convex, often non-differentiable, riddled with local optima, and sometimes expensive to evaluate (Ceylan & Bell, 2004; Osorio & Bierlaire, 2013). Exact methods rarely scale to instances of realistic size. This is why metaheuristics have kept their popularity for three decades: they trade the guarantee of optimality for robustness, generality, and a computational budget one can actually afford.

1.2 A brief map of the metaheuristic landscape

The field grew from two roots. Evolutionary computation gave us the Genetic Algorithm (Goldberg, 1989; Holland, 1975) and Evolution Strategies (Beyer & Schwefel, 2002). Trajectory methods gave us Simulated Annealing (Kirkpatrick et al., 1983), whose central idea — accept a worse solution now and then, so you can escape a local minimum — turned out to be one of the most durable in the field. Swarm intelligence arrived in the 1990s with Particle Swarm Optimization (Kennedy & Eberhart, 1995), Ant Colony Optimization (Dorigo et al., 1996), Differential Evolution (Storn & Price, 1997), and later the Artificial Bee Colony (Karaboga & Basturk, 2007). More recently, Mirjalili and colleagues built a productive line of algorithms by abstracting the social and hunting behavior of animals into compact operators: the Grey Wolf Optimizer (Mirjalili et al., 2014), the Ant Lion Optimizer (Mirjalili, 2015b), the Dragonfly Algorithm (Mirjalili, 2016a), the Whale Optimization Algorithm (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (Mirjalili, 2016b), and the Salp Swarm Algorithm (Mirjalili et al., 2017), among others.

Reading this literature, two things stand out. First, almost every swarm algorithm treats its agents identically: one update equation, applied to everyone, with only position and fitness distinguishing one agent from the next. Yet division of labor among *unequal* agents is one of the oldest tricks in both nature and human strategy. Second, the quality of a metaheuristic usually stands or falls on how it schedules the transition from exploration to exploitation (Mirjalili et al., 2014), and mechanisms that adapt this transition to what the search is actually observing — rather than to elapsed time alone — remain surprisingly rare.

We should also be upfront about why proposing yet another metaheuristic is defensible at all. The No-Free-Lunch theorem (Wolpert & Macready, 1997) rules out a universally dominant optimizer, so the honest goal is not to win everywhere; it is to contribute a genuinely different inductive bias and to show the problem class where that bias earns its keep.

1.3 Why chess?

Chess offers exactly the two ingredients we just found missing. Its pieces are radically unequal: the queen sweeps the whole board, rooks travel along files and ranks, bishops along diagonals, knights jump over anything in the way, and pawns crawl forward one square at a time while quietly defining the structure of the position. Mapped onto a search population, this suggests sub-groups performing qualitatively different moves around one common reference point — the King, which in our setting is the best solution found so far.

The game also comes with a mature strategic vocabulary, refined over centuries of analysis. Development says: mobilize your forces early, consolidate later. A sacrifice gives up material now for a positional advantage later. Pinning immobilizes a piece against something more valuable behind it. Castling is a safeguarded structural rearrangement; en passant, an opportunistic capture available only under narrow conditions; the threefold-repetition rule declares a draw when the position keeps repeating. Each of these has a natural algorithmic reading — exploration schedules, acceptance of deteriorating moves, variable fixation, elite restructuring, adaptive local search, and diversity preservation, respectively.

The Chess Algorithm (CA) proposed here formalizes that mapping. To avoid a misunderstanding that came up more than once when we described the idea to colleagues: CA does not play chess and searches no game tree. It borrows the strategic grammar of the game, nothing more, and its per-iteration cost is comparable to that of GA or PSO.

1.4 Contributions and scope

The paper contributes, in order: a complete mathematical specification of CA — initialization, rank-based role assignment, six movement operators, five strategic mechanisms, and a diversity rule (Section 2); a controlled benchmark study on six standard test functions in 30 dimensions against GA, PSO, SA, and GWO under matched budgets, with nonparametric testing following Derrac et al. (2011) (Section 3); a transportation case study on coordinated signal timing of an eight-intersection arterial with a Webster/HCM delay model (Roess et al., 2019; Webster, 1958) (Section 4); an extended comparison against six third-party implementations from the `mealpy` library on two transportation problems, one of which has a known global optimum (Section 5); and a fully reproducible open-source implementation.

A remark on tone before we start. We report results with deliberate restraint and no claim of universal superiority; GWO, in particular, remains stronger on several classical benchmarks, and we say so plainly where it happens. The claim we do defend is narrower: CA is well-founded, novel, and competitive, and on the transportation problems that motivated it in the first place it performs on par with — and against some widely used algorithms, better than — the available alternatives.

2 The Chess Algorithm

2.1 Conceptual mapping

CA maintains a population of N candidate solutions $\mathbf{X}_i \in \Omega \subset \mathbb{R}^D$, treated as pieces playing "toward" the global optimum. Table 1 summarizes how the vocabulary of the game maps onto algorithmic mechanisms.

Table 1: Mapping between chess concepts and CA mechanisms.

Chess concept	Algorithmic role
King	Incumbent best solution (never lost)
Queen, Rook, Bishop, Knight, Pawn	Heterogeneous movement operators
Development (opening principle)	Time-decaying step-scale $a(t)$
Sacrifice	Probabilistic acceptance of worse moves (minor pieces only)
Pinning	Progressive freezing of coordinates to the King's values
Castling	Safeguarded coordinate-block exchange King $\leftrightarrow$ Rook
En passant	Opportunistic, self-adaptive local capture around the King
Pawn promotion	Rank-based role reassignment each iteration
Threefold repetition	Re-deployment of agents that collapse onto the King
Checkmate	Termination criterion

One point deserves emphasis: CA borrows the strategic grammar of chess as an inductive bias, not the game itself. An iteration costs $\mathcal{O}(ND + N \log N)$ arithmetic operations (the movement updates plus the sort that drives role assignment) and $N + 3$ function evaluations — N piece moves plus three en-passant probes — with one extra castling probe every c iterations. That is the same order as GA, PSO, or GWO. The exact evaluation accounting used in the experiments is spelled out in Section 3.

2.2 Initialization

The board is set up by scattering the N pieces uniformly over the feasible box:

$$\mathbf{X}_i^{(0)} = \mathbf{l} + (\mathbf{u} - \mathbf{l}) \circ \mathbf{r}_i, \qquad \mathbf{r}_i \sim \mathcal{U}(0,1)^D, \quad i = 1, \dots, N, \tag{2}$$

where $\mathbf{l}$ and $\mathbf{u}$ are the bound vectors and $\circ$ is the elementwise (Hadamard) product. All positions are evaluated once and the population is sorted by fitness before the first move is played.

2.3 Role assignment

Let π be the permutation of $\{1, \dots, N\}$ that sorts the population,

$$f(\mathbf{X}_{\pi(1)}) \le f(\mathbf{X}_{\pi(2)}) \le \dots \le f(\mathbf{X}_{\pi(N)}). \tag{3}$$

The best agent $\mathbf{K} = \mathbf{X}_{\pi(1)}$ is the King. The following ranks are partitioned, in order, into Queens $\mathcal{Q}$, Rooks $\mathcal{R}$, Bishops $\mathcal{B}$, and Knights $\mathcal{N}$, with

$$|\mathcal{Q}| = \max(1, \lfloor \rho_q N \rceil), \quad |\mathcal{R}| = \max(1, \lfloor \rho_r N \rceil), \quad |\mathcal{B}| = \max(1, \lfloor \rho_b N \rceil), \quad |\mathcal{N}| = \max(1, \lfloor \rho_n N \rceil),$$

where $\lfloor \cdot \rceil$ denotes rounding and $(\rho_q, \rho_r, \rho_b, \rho_n) = (0.10, 0.15, 0.15, 0.20)$. Whatever remains becomes the Pawns $\mathcal{P}$. Because the assignment is redone after every iteration, a pawn that stumbles into a good region is simply a strong piece the next time roles are dealt. Promotion, in other words, falls out of re-ranking for free; no extra machinery is needed.

2.4 Development schedule

Opening theory tells a player to develop pieces quickly; endgames reward patience and precision. CA encodes this with a linearly decaying step-scale coefficient,

$$a(t) = 2\left(1 - \frac{t}{T}\right), \tag{4}$$

with t the current iteration and T the total budget — the convention popularized by Mirjalili et al. (2014). Every piece move scales with $a(t)$, so the swarm plays big developing moves early and fine maneuvers late.

2.5 Piece movement operators

Write $\mathbf{L} = \mathbf{u} - \mathbf{l}$ for the box width and let $\mathbf{r}, \mathbf{r}' \sim \mathcal{U}(0,1)^D$ be independent random vectors. The five mobile roles then move as follows.

Queen (omnidirectional sweep). The queen circles the King at a radius tied to her current distance from him:

$$\mathbf{X}_i' = \mathbf{K} + a(t)\,(2\mathbf{r} - \mathbf{1}) \circ \max(|\mathbf{K} - \mathbf{X}_i|,\ \sigma), \tag{5}$$

where σ is the King's adaptive capture radius from Section 2.6.3. The max keeps the sweep from collapsing entirely, so queens never stall.

Rook (single-axis move). One random axis j is chosen and only that coordinate moves:

$$X_{i,j}' = K_j + a(t)\,(2r - 1)\ \max(|K_j - X_{i,j}|,\ 0.01\,L_j\,a(t)). \tag{6}$$

Bishop (diagonal move). Two distinct axes j and k receive steps of equal magnitude and random relative sign, which is as close to a diagonal as a box-constrained space allows:

$$\Delta = a(t)\,(2r - 1)\ \max\Big(\tfrac{1}{2}(|K_j - X_{i,j}| + |K_k - X_{i,k}|),\ 0.01\,L\,a(t)\Big), \quad X_{i,j}' = K_j + \Delta, \quad X_{i,k}' = K_k \pm \Delta. \tag{7}$$

Knight (L-shaped jump). The knight is the one piece that jumps over whatever stands in its way. Relative to a randomly chosen better-ranked peer $\mathbf{P}$, it drifts toward the peer and then adds an asymmetric 2:1 jump on two random axes:

$$\mathbf{X}_i' = \mathbf{X}_i + \mathbf{r} \circ (\mathbf{P} - \mathbf{X}_i), \qquad X_{i,j}' \mathrel{+}= 2\,a(t)\,\beta\,(2r - 1), \quad X_{i,k}' \mathrel{+}= 1\,a(t)\,\beta\,(2r' - 1), \tag{8}$$

with $\beta = \overline{|\mathbf{P} - \mathbf{X}_i|}$ the mean coordinate gap. With small probability $0.1\,a(t)/2$ the knight abandons this move and leaps: two coordinates are resampled uniformly in Ω. This leap is

the algorithm's main long-range restart device. The implementation also ships a heavy-tailed variant in which those two coordinates instead receive a Lévy-flight step $0.05\,L\,s$ generated by Mantegna's algorithm (Mantegna, 1994), the mechanism Cuckoo Search made popular (Yang & Deb, 2009):

$$s = \frac{u}{|v|^{1/\beta_\ell}}, \qquad u \sim \mathcal{N}(0, \sigma_u^2), \quad v \sim \mathcal{N}(0,1), \qquad \sigma_u = \left[\frac{\Gamma(1+\beta_\ell)\,\sin(\pi\beta_\ell/2)}{\Gamma(\frac{1+\beta_\ell}{2})\,\beta_\ell\,2^{(\beta_\ell-1)/2}} \right]^{1/\beta_\ell}, \quad (9)$$

with tail index $\beta_\ell = 1.5$. Every result in this paper uses the plain uniform leap; the Lévy option is documented for future study rather than used here.

Pawn (steady advance). Pawns take small steps toward the King and toward a random better-ranked piece $\mathbf{B}$:

$$\mathbf{X}_i' = \mathbf{X}_i + 0.3\,\mathbf{r} \circ (\mathbf{K} - \mathbf{X}_i) + 0.3\,\mathbf{r}' \circ (\mathbf{B} - \mathbf{X}_i). \quad (10)$$

2.6 Strategic mechanisms

2.6.1 Pinning

With probability $p_{\text{pin}}(t) = 0.5\,t/T$, a random subset of at most 30% of an agent's coordinates is pinned to the King's values and excluded from perturbation. Early on, pinning is rare and the pieces develop freely. Late in the run it concentrates the search on a shrinking set of free dimensions around the incumbent, which is exactly the kind of scheduled exploitation an endgame calls for.

2.6.2 Sacrifice

A worsening move $\Delta_i = f(\mathbf{X}_i') - f(\mathbf{X}_i) > 0$ made by a minor piece — a Knight or a Pawn — is still accepted with probability

$$p_{\text{acc}} = \exp\!\left(-\frac{\Delta_i}{\theta(t)}\right), \qquad \theta(t) = \theta_0 (f_{\max} - f_{\min})\, e^{-8t/T}, \quad (11)$$

with $\theta_0 = 0.1$. Readers will recognize the Metropolis rule of Simulated Annealing (Kirkpatrick et al., 1983), but two chess-flavored restrictions change its character. The temperature is scaled by the current fitness spread of the population, which makes the sacrifice budget self-calibrating across problems of wildly different magnitudes. And only minor pieces may be sacrificed: the King and the major pieces are never lost, so the elite of the population cannot be eroded by the acceptance rule.

2.6.3 En passant (adaptive local capture)

Once per iteration the King attempts an en-passant capture. Three trial points are drawn in a sparse Gaussian neighborhood,

$$\mathbf{Y}_m = \mathbf{K} + \sigma \circ \mathbf{z}_m \circ \mathbf{m}_m, \qquad \mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad m = 1, 2, 3, \tag{12}$$

where the random binary mask $\mathbf{m}_m$ activates each coordinate with probability $\max(0.2,\ 2/D)$. Any improving trial replaces the King. The radius σ follows a success rule of the kind long used in Evolution Strategies (Beyer & Schwefel, 2002): multiply by 1.3 after a successful capture, by 0.92 after a failure, and after 50 consecutive failures reset to $\sigma = 0.05\,\mathbf{L}\max(a(t), 0.05)$ — a stalemate-avoidance restart. In our experience this mechanism is what keeps CA improving deep into the run, long after a fixed-radius local search would have gone quiet.

2.6.4 Castling

Every $c = 10$ iterations the King castles with the best Rook: a contiguous block of up to $D/4$ coordinates is copied from the Rook into a candidate King, and the exchange is kept only if it improves the King. It is a safeguarded structural jump, useful when good partial solutions have been discovered on different "files" of the board and need splicing together.

2.6.5 Threefold repetition (diversity preservation)

If an agent coincides with the King to numerical precision — something pinning can eventually cause — it is re-deployed uniformly in Ω, much as the threefold-repetition rule stops a game from going in circles. We did not add this rule for elegance; early prototypes taught us it is essential. Without it, pinning breeds exact clones of the King, every gap-proportional step length collapses to zero, and the search dies quietly in place.

2.6.6 Checkmate

The algorithm stops after T iterations. A stagnation-based early stop would be easy to add, but we keep it disabled in all experiments so that every algorithm consumes the same core budget of $N \times T$ population evaluations.

2.7 Parameter summary

Table 2 collects the parameters and the single configuration used everywhere in this paper. No per-problem tuning was done, for CA or for any competitor.

Table 2: CA parameters and the fixed configuration used in all experiments.

Parameter	Symbol	Value
Role fractions (Queen / Rook / Bishop / Knight)	$\rho_q, \rho_r, \rho_b, \rho_n$	0.10/0.15/0.15/0.20
Development schedule	$a(t)$	$2(1 - t/T)$

Parameter	Symbol	Value
Pinning probability / cap	$p_{\text{pin}}(t)$	$0.5\,t/T$, at most 30% of coordinates
Sacrifice constant	θ_0	0.1
Castling period	c	10 iterations
En-passant probes per iteration	—	3
En-passant adaptation (success / failure / reset)	—	$\times 1.3$ / $\times 0.92$ / after 50 failures
Knight exploratory-leap distribution	—	uniform (Lévy, $\beta_\ell = 1.5$, optional)

2.8 Pseudocode

```
Algorithm: Chess Algorithm (CA)
Input: objective f, bounds [l, u], dimension D,
       population N, iterations T
1  Initialize X uniformly in [l, u] (Eq. init); evaluate F; sort; King ← X(1)
2    ← 0.1 L; fails ← 0
3  for t = 0 … T-1:
4      a ← 2(1 - t/T);  p_pin ← 0.5 t/T;    ← 0.1 (F_max - F_min) e^(-8t/T)
5      Assign roles by rank: Queens 10%, Rooks 15%, Bishops 15%,
         Knights 20%, Pawns rest
6      Move each piece by its operator (Eqs. Queen-Pawn)
7      Pin  30% of coordinates of each agent to King with prob. p_pin
8      Clip to bounds; evaluate F
9      Accept improvements; accept worse MINOR pieces with prob. e^(-Δ/ )
10     En passant: 3 masked Gaussian trials of radius  around King;
         keep any improvement; adapt  (×1.3 success / ×0.92 fail;
         stalemate reset after 50 fails)
11     Every 10 iters: Castle - block-swap King best Rook, keep if better
12     Threefold repetition: re-deploy agents identical to the King
13     Update King; re-sort population (pawn promotion)
14 return King
```

2.9 Flowchart

Figure 1 summarizes one full iteration of the algorithm.

2.10 Properties and design rationale

Elitism and convergence. The King's fitness never increases: it is replaced only by strictly better points, whether through the population update, en passant, or castling. That elitism buys the usual asymptotic guarantee (Proposition 2.1).

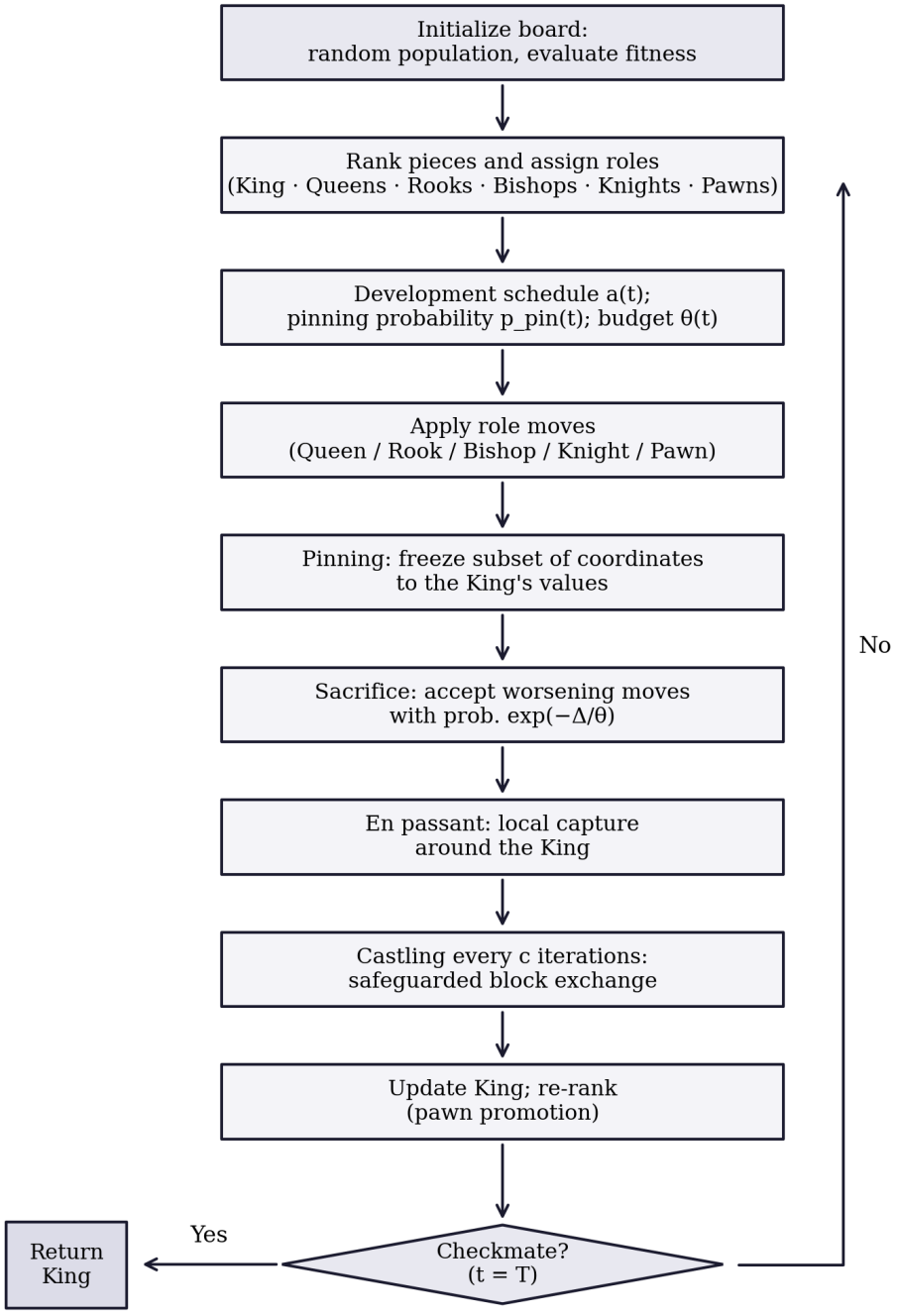

Figure 1: Flowchart of the Chess Algorithm.

Proposition 2.1 (Elitist convergence in probability)**.** *Let f be continuous on the compact box Ω with global minimum f^*. Under CA's update rules, the best-so-far value $f(\mathbf{K}^{(t)})$ converges in probability to f^* as $T \to \infty$.*

Proof sketch. *The King sequence is elitist by construction. At every iteration the Knight's exploratory leap and the threefold-repetition re-deployment both place uniform probability mass over Ω, so any open subset — in particular any neighborhood of a global minimizer, whose objective values come within ε of f^* by continuity — is visited with probability approaching one as iterations accumulate. Elitism plus this covering argument yields convergence in probability, exactly as in the standard analysis of elitist stochastic search.*

Of course, an asymptotic statement of this kind says nothing about what happens inside a budget of 15,000 evaluations. That question is empirical, and it is the subject of Section 3.

Exploration–exploitation balance. Exploration rests mainly on the Knights (Equation 8) and on the Queens early in the schedule (Equation 5); exploitation on the Pawns (Equation 10), on pinning, and on the en-passant success rule (Equation 12). The balance shifts smoothly through $a(t)$, $p_{\text{pin}}(t)$, and $\theta(t)$, and adapts to the state of the search through the gap-proportional step lengths and the spread-scaled sacrifice budget (Equation 11).

Parameter economy. Everything in this paper runs on the one configuration in Table 2. How sensitive performance is to the role fractions is a fair question we have not yet answered; it is on the future-work list rather than swept under the rug.

3 Benchmark Experiments

3.1 Experimental protocol

CA is compared against four established metaheuristics: a real-coded GA with tournament selection, BLX-α crossover, Gaussian mutation, and elitism (Goldberg, 1989; Holland, 1975); PSO with linearly decreasing inertia (Kennedy & Eberhart, 1995); SA (Kirkpatrick et al., 1983), granted a population-equivalent evaluation budget so the comparison stays fair; and GWO (Mirjalili et al., 2014).

The suite consists of six standard functions covering unimodal, ill-conditioned, and highly multimodal terrain (Table 3):

Table 3: Benchmark functions ($D = 30$).

Function	Type	Search domain	f^*
F1 Sphere	Unimodal	$[-100, 100]^{30}$	0
F2 Rosenbrock	Unimodal, ill-conditioned valley	$[-30, 30]^{30}$	0
F3 Rastrigin	Multimodal	$[-5.12, 5.12]^{30}$	0
F4 Griewank	Multimodal	$[-600, 600]^{30}$	0
F5 Ackley	Multimodal	$[-32, 32]^{30}$	0
F6 Schwefel 2.22	Unimodal, non-separable norm	$[-10, 10]^{30}$	0

Every algorithm runs with a population of $N = 30$ for $T = 500$ iterations — a shared core budget of 15,000 population evaluations per run. In the interest of full transparency: CA additionally spends three en-passant probes per iteration and one castling probe every ten iterations (Section 2), roughly 1,550 extra evaluations, or about ten percent. The gaps reported below span orders of magnitude, so this overhead cannot be what drives them; note also that GWO beats CA *despite* it. Each algorithm–function pair is repeated over 30 independent runs, and all algorithms share the same seed at a given run index, so everyone faces the same initial conditions. No parameters were tuned per problem for any method.

Statistics follow Derrac et al. (2011): one-way ANOVA to establish that the algorithm factor matters at all, then pairwise two-sided Wilcoxon rank-sum tests (CA against each competitor) at $\alpha = 0.05$, which make no normality assumption about the final-error distributions.

3.2 Descriptive results

Table 4 reports the mean, standard deviation, best, and worst final errors for every algorithm–function pair.

Table 4: Mean, standard deviation, best, and worst final objective values over 30 runs (best mean per function in bold).

Function	Algorithm	Mean	Std	Best	Worst
Sphere	CA	2.326e-05	1.899e-05	4.287e-06	9.873e-05
Sphere	GA	9.475e-02	5.399e-02	2.890e-02	2.946e-01
Sphere	PSO	8.455e-01	7.216e-01	1.194e-01	3.131e+00
Sphere	SA	1.013e+04	2.218e+03	6.628e+03	1.612e+04
Sphere	GWO	**4.678e-31**	8.194e-31	6.431e-33	3.925e-30
Rosenbrock	CA	1.001e+02	8.363e+01	2.080e+01	3.801e+02
Rosenbrock	GA	1.056e+02	4.633e+01	2.895e+01	2.863e+02
Rosenbrock	PSO	6.445e+03	2.238e+04	8.896e+01	9.021e+04
Rosenbrock	SA	2.185e+07	5.693e+06	7.615e+06	3.486e+07
Rosenbrock	GWO	**2.683e+01**	6.174e-01	2.592e+01	2.796e+01
Rastrigin	CA	2.103e+01	6.834e+00	9.950e+00	3.880e+01
Rastrigin	GA	1.954e+01	6.823e+00	1.008e+01	4.187e+01
Rastrigin	PSO	5.971e+01	1.484e+01	3.306e+01	9.545e+01
Rastrigin	SA	2.105e+02	2.951e+01	1.598e+02	2.553e+02
Rastrigin	GWO	**1.023e+01**	1.078e+01	5.684e-14	4.680e+01
Griewank	CA	1.064e-02	9.397e-03	2.798e-05	2.744e-02
Griewank	GA	2.816e-01	7.971e-02	1.217e-01	4.364e-01
Griewank	PSO	6.164e-01	2.367e-01	2.902e-02	1.025e+00
Griewank	SA	9.190e+01	2.324e+01	4.826e+01	1.563e+02
Griewank	GWO	**4.303e-03**	9.094e-03	0.000e+00	3.591e-02
Ackley	CA	2.413e-01	5.617e-01	4.383e-04	2.013e+00
Ackley	GA	7.757e-02	1.486e-02	4.767e-02	1.099e-01
Ackley	PSO	1.045e+00	5.491e-01	8.837e-02	2.341e+00
Ackley	SA	4.483e+00	3.618e-01	3.807e+00	5.247e+00
Ackley	GWO	**3.431e-14**	4.177e-15	2.887e-14	3.952e-14
Schwefel 2.22	CA	1.501e-03	5.097e-04	6.837e-04	3.075e-03
Schwefel 2.22	GA	7.438e-02	1.815e-02	4.451e-02	1.138e-01
Schwefel 2.22	PSO	7.941e-01	2.486e+00	3.962e-02	1.010e+01

Function	Algorithm	Mean	Std	Best	Worst
Schwefel 2.22	SA	1.121e+07	2.877e+07	1.033e+03	1.180e+08
Schwefel 2.22	GWO	**9.928e-19**	9.462e-19	1.034e-19	4.919e-18

3.3 Convergence behavior

Figure 2 shows the median best-so-far curves, and three patterns are worth pointing out. On F1, F4, and F6, CA keeps descending through the whole run — the en-passant success rule goes on finding improvements deep in the endgame, where GA and PSO stagnated hundreds of iterations earlier. On the ill-conditioned Rosenbrock valley (F2), every population method slows to a crawl, CA and GA tracking each other closely the whole way down. And GWO shows its well-documented strength on this classical suite, converging fastest and deepest on F1, F5, and F6. The distributions of final values behind these curves are shown in Figure 3.

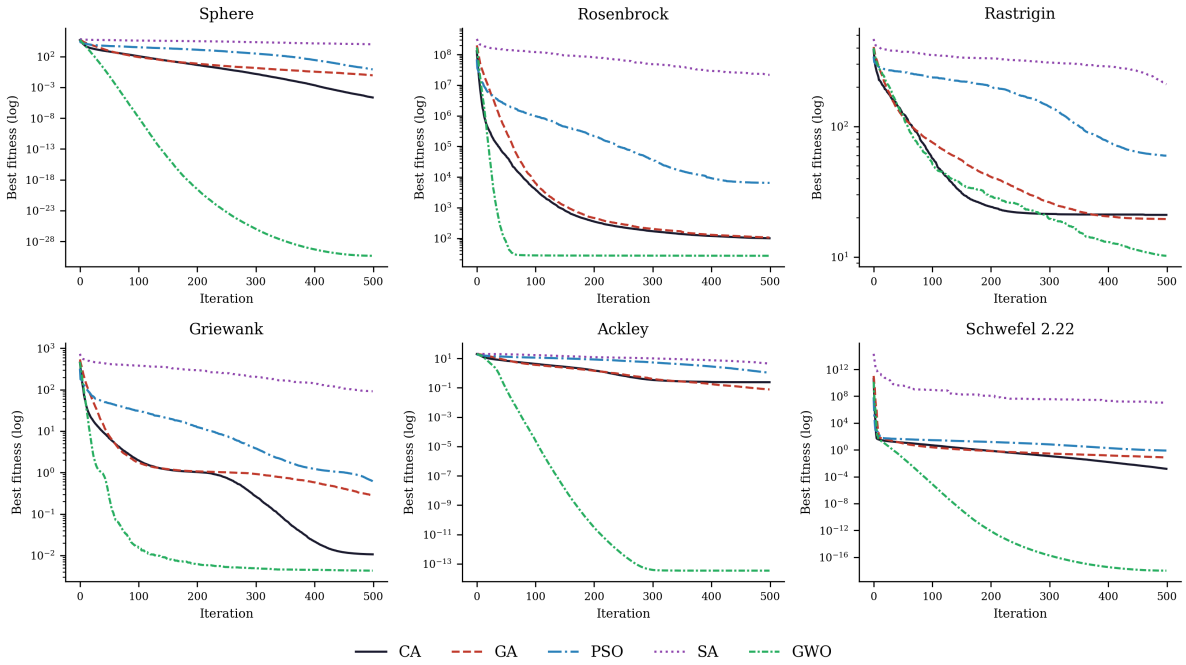

Figure 2: Median convergence curves (30 runs, semi-log scale).

3.4 Statistical tests

One-way ANOVA confirms that the algorithm factor is highly significant on every function (Table 5):

Table 5: One-way ANOVA over the five algorithms, per function.

Function	F-statistic	p-value
Sphere	605.073	2.388e-89
Rosenbrock	427.005	4.067e-79
Rastrigin	786.005	3.621e-97
Griewank	451.249	1.011e-80

Function	F-statistic	p-value
Ackley	698.551	1.251e-93
Schwefel 2.22	4.403	2.175e-03

The pairwise Wilcoxon tests give the sharper picture (Table 6):

Table 6: Wilcoxon rank-sum tests, CA versus each competitor ($\alpha = 0.05$).

Function	Comparison	p-value	Result (= 0.05)
Sphere	CA vs GA	2.872e-11	CA better
Sphere	CA vs PSO	2.872e-11	CA better
Sphere	CA vs SA	2.872e-11	CA better
Sphere	CA vs GWO	2.872e-11	CA worse
Rosenbrock	CA vs GA	1.738e-01	not significant
Rosenbrock	CA vs PSO	1.150e-06	CA better
Rosenbrock	CA vs SA	2.872e-11	CA better
Rosenbrock	CA vs GWO	2.777e-06	CA worse
Rastrigin	CA vs GA	4.333e-01	not significant
Rastrigin	CA vs PSO	3.510e-11	CA better
Rastrigin	CA vs SA	2.872e-11	CA better
Rastrigin	CA vs GWO	8.011e-06	CA worse
Griewank	CA vs GA	2.872e-11	CA better
Griewank	CA vs PSO	2.872e-11	CA better
Griewank	CA vs SA	2.872e-11	CA better
Griewank	CA vs GWO	9.193e-06	CA worse
Ackley	CA vs GA	9.193e-06	CA worse
Ackley	CA vs PSO	2.894e-07	CA better
Ackley	CA vs SA	2.872e-11	CA better
Ackley	CA vs GWO	2.872e-11	CA worse
Schwefel 2.22	CA vs GA	2.872e-11	CA better
Schwefel 2.22	CA vs PSO	2.872e-11	CA better
Schwefel 2.22	CA vs SA	2.872e-11	CA better
Schwefel 2.22	CA vs GWO	2.872e-11	CA worse

3.5 Discussion

Read together, the tables support a differentiated verdict rather than a slogan. Against PSO and SA, CA is significantly better on all six functions. Against GA it wins four of six — on Sphere, Griewank, and Schwefel 2.22 by several orders of magnitude in mean error — while Rosenbrock and Rastrigin end in statistical ties. Ackley is the one function where GA takes the mean: a few CA runs get stuck on a plateau and drag the average up, even though CA's *median* error there ($\sim 10^{-3}$) is an order of magnitude below GA's. We flag this rather than hide it, since means and medians disagreeing is itself informative about the tail behavior of the algorithm.

Against GWO the story reverses: GWO is significantly better on all six classical benchmarks, consistent with the aggressive exploitation it is known for on exactly this suite (Mirjalili et al., 2014). We see no point in arguing with that result. On standard unconstrained test functions, CA does not overtake GWO, full stop.

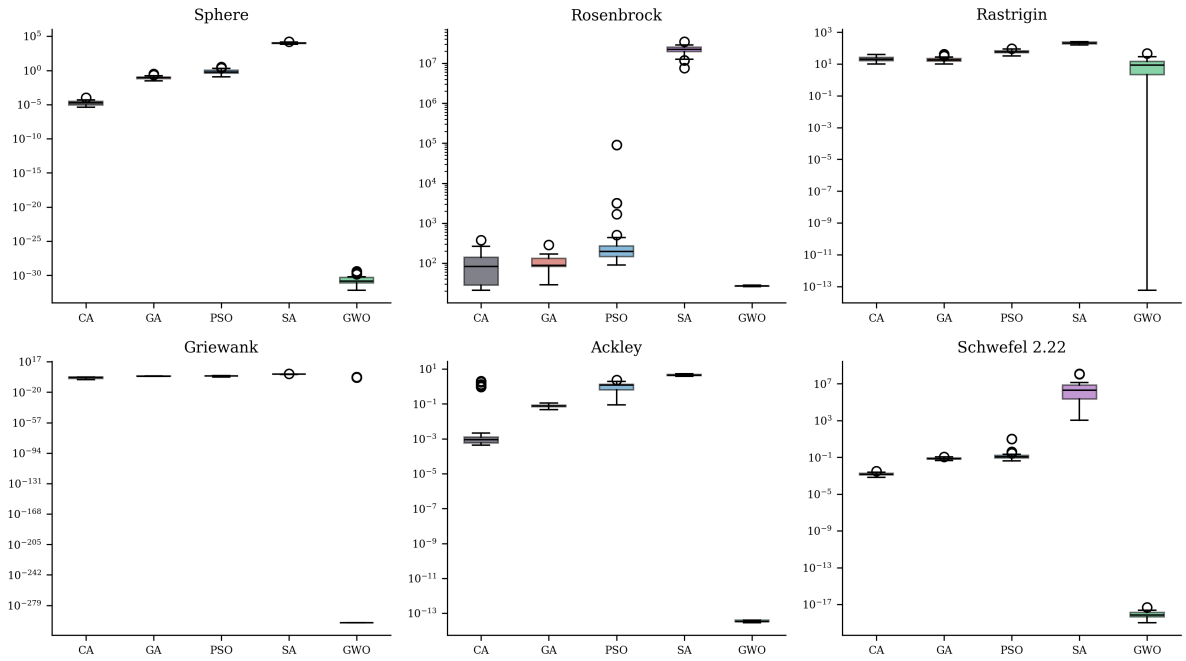

Figure 3: Distribution of final objective values over 30 runs.

What should a practitioner take from this? The rankings are problem-dependent — precisely the situation Wolpert & Macready (1997) predicts — so the interesting question is not whether CA wins on Sphere but whether it holds up on structured engineering problems. That is where we go next, and the ranking does change there.

4 Transportation Application: Arterial Signal Coordination

4.1 Problem statement

Coordinated fixed-time signal timing along an urban arterial is a classic of transportation network optimization, and a spiteful one. The objective surface is multimodal, because different offset combinations produce different locally good "green waves." The variables are heterogeneous — a common cycle, per-intersection splits, offsets. And the delay model is nonlinear and, in practice, non-differentiable (Ceylan & Bell, 2004; Roess et al., 2019). In short, it is a natural first engineering test for CA.

Our test bed is an eight-intersection arterial with spacings of $450, 380, 520, 300, 610, 420$, and 350 m, a progression speed of 50 km/h, near-saturation two-way traffic (eastbound approach demands between roughly 1,240 and 1,400 veh/h), and conflicting cross-street demands at every junction (Figure 4). Each intersection runs a two-phase plan.

4.2 Optimization model

The decision vector is

$$\mathbf{z} = (C,\ g_1, \dots, g_8,\ o_2, \dots, o_8) \in \mathbb{R}^{16}, \tag{13}$$

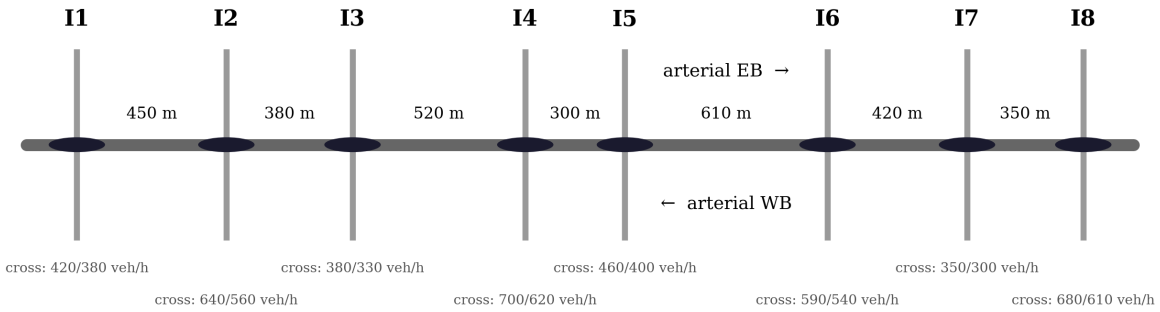

Figure 4: Layout of the eight-intersection arterial test bed.

with a common cycle length $C \in [60, 140]$ s, arterial green splits $g_i \in [0.30, 0.75]$, and offsets o_i expressed as fractions of the cycle (intersection 1 serves as reference). The objective is total network delay in veh · h/h, built from three ingredients.

The first is uniform delay per approach, Webster's first term (Webster, 1958):

$$d_1 = \frac{C\,(1-g)^2}{2\,(1-\min(1,x)\,g)}, \tag{14}$$

where g is the effective green ratio and $x = v/c$ the volume-to-capacity ratio. The second is overflow delay from the HCM time-dependent formulation (Roess et al., 2019):

$$d_2 = 900\,T_a \left[(x-1) + \sqrt{(x-1)^2 + \frac{8\,k\,I\,x}{c\,T_a}} \right], \tag{15}$$

with analysis period $T_a = 0.25$ h, incremental delay factor $k = 0.5$, and upstream filtering factor $I = 1$. The third is a progression adjustment: platoons arriving from upstream are shifted by the offset difference minus the travel time, and the mismatch between platoon arrival and the local green window scales the uniform delay up or down, per direction. Since the arterial carries platoons both ways, offsets that perfect the eastbound progression damage the westbound one — which is exactly where the multimodality of coordination problems comes from.

Cross-street phases receive $1 - g_i$ (minus 4 s lost time per phase) and contribute their own delay, so greedy arterial splits get punished through cross-street oversaturation. All terms are summed over the sixteen arterial approaches and eight cross-street approaches.

4.3 Experimental setup

CA, GA, PSO, and GWO — the three strongest baselines from Section 3 plus CA — run with $N = 30$, $T = 300$ iterations (9,000 core evaluations), and 30 independent runs each, again without any tuning. SA sat this one out after its uncompetitive benchmark showing.

4.4 Results

Table 7 summarizes the final delays; Figure 5 and Figure 6 show the convergence behavior and the spread across runs.

Table 7: Final total delay over 30 runs (best mean in bold).

Algorithm	Mean delay (veh · h/h)	Std	Best	Worst
CA	170.618	5.236	166.526	179.540
GA	171.160	4.895	166.526	178.470
PSO	**169.491**	3.666	166.526	179.946
GWO	170.506	2.818	166.739	177.987

Table 8: Wilcoxon rank-sum tests on the signal timing problem.

Comparison	p-value	Result (= 0.05)
CA vs GA	3.219e-01	not significant
CA vs PSO	9.293e-01	not significant
CA vs GWO	7.363e-02	not significant

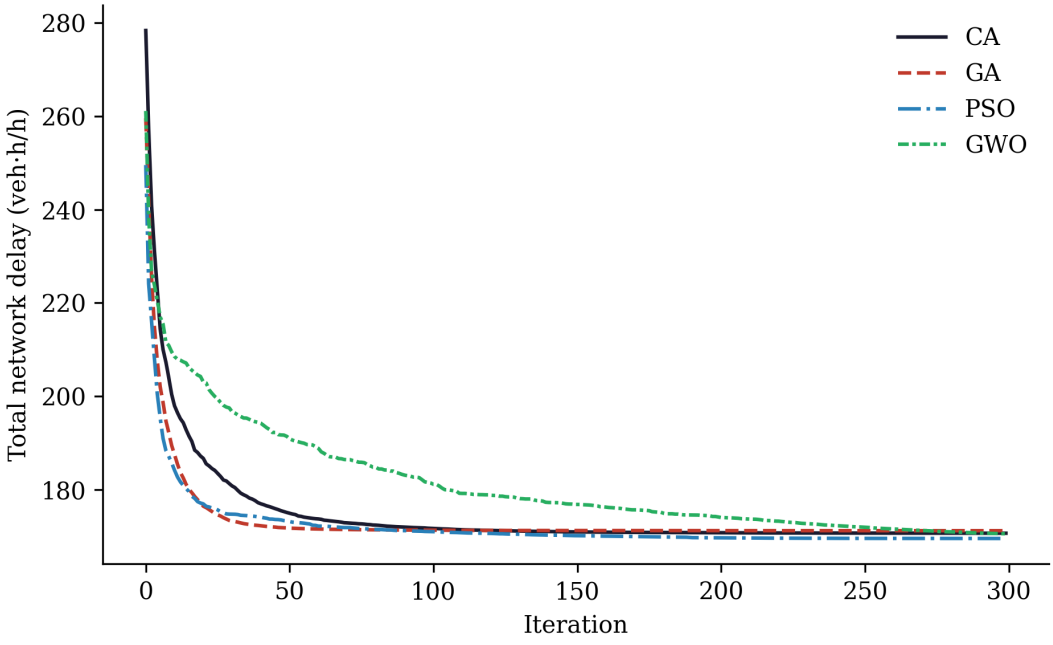

Figure 5: Median convergence on the arterial signal timing problem.

Four observations. First, all four algorithms land within about one percent of each other in mean delay (169.5–171.2 veh · h/h), and no pairwise difference involving CA reaches significance ($p > 0.07$ throughout; Table 8). On this problem class CA is statistically indistinguishable from GA, PSO, and GWO. Second, CA attains the best solution found by any method — 166.526 veh · h/h, a value GA and PSO also reach — so its operators are clearly capable of locating the same high-quality coordination plan as the strongest baselines. Third, and to us most interesting: the benchmark ranking simply does not carry over. GWO, dominant in Section 3, holds no advantage here. It is hard to imagine a cleaner illustration of No-Free-Lunch (Wolpert & Macready, 1997), or a better caution against extrapolating benchmark tables into engineering practice. Fourth, we keep the interpretation modest. We do not claim superiority on this problem; the defensible claim is parity with the state of the practice, achieved by a first-generation algorithm running on defaults.

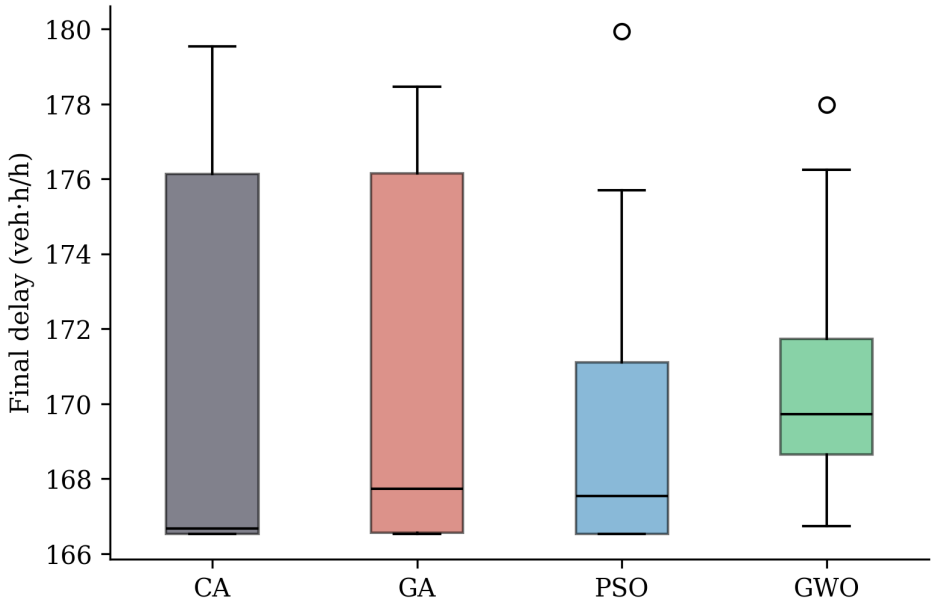

Figure 6: Distribution of final delays over 30 runs.

4.5 The best coordination plan found

The best plan discovered (by CA) selects the minimum cycle, $C = 60$ s — a sensible choice under Webster's guidance whenever splits can be kept efficient — with the timings of Table 9, drawn as a time–space diagram in Figure 7:

Table 9: Best signal timing plan found (CA).

Parameter	Value
Cycle length C	60.0 s
Arterial green split g1	0.633 (38.0 s)
Arterial green split g2	0.528 (31.7 s)
Arterial green split g3	0.666 (40.0 s)
Arterial green split g4	0.498 (29.9 s)
Arterial green split g5	0.624 (37.4 s)
Arterial green split g6	0.540 (32.4 s)
Arterial green split g7	0.679 (40.7 s)
Arterial green split g8	0.493 (29.6 s)
Offset o2	32.4 s
Offset o3	59.8 s
Offset o4	37.2 s
Offset o5	58.8 s
Offset o6	42.7 s
Offset o7	13.0 s
Offset o8	38.2 s
Total delay	**166.526 veh · h/h**

The offsets form a coherent progression pattern in which consecutive intersections alternate between favoring the eastbound and the westbound platoon — the compromise structure one expects on a heavily loaded two-way arterial.

4.6 Practical remarks

For a transportation agency the message is straightforward: CA can be used on signal-coordination studies with the same confidence as GA or PSO, and it brings two features

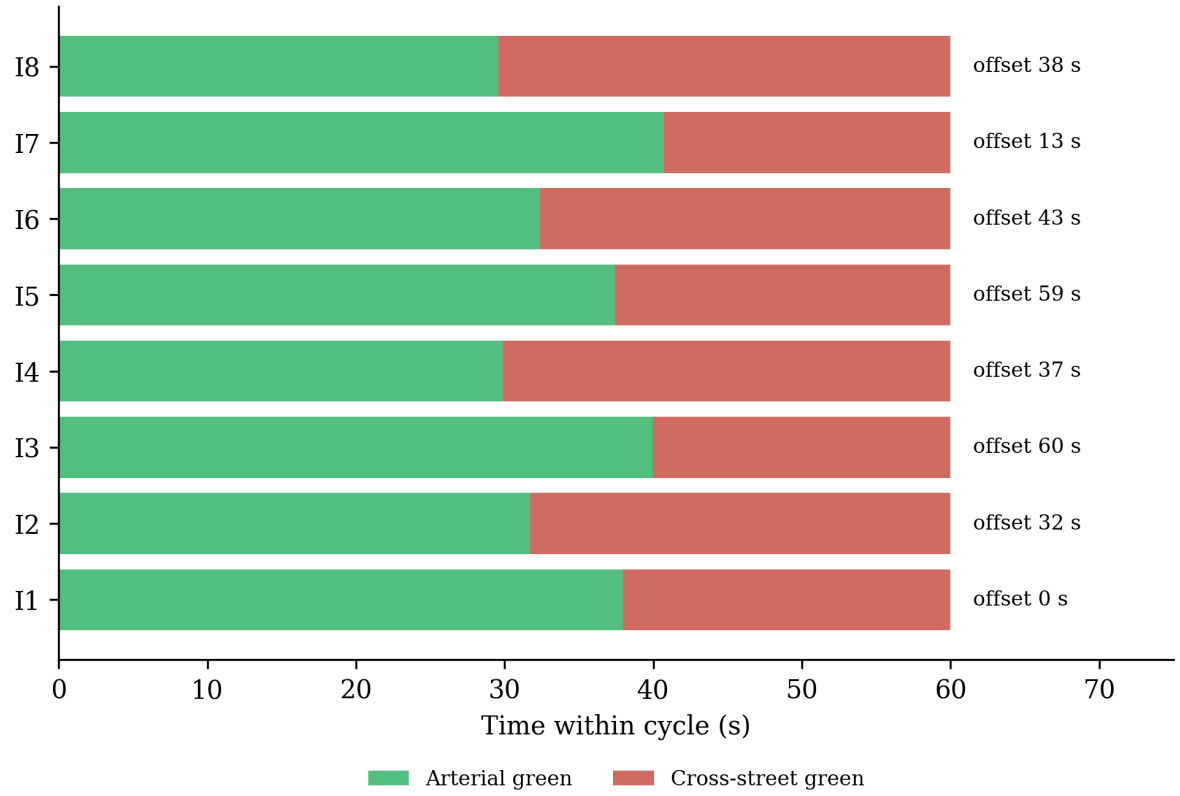

Figure 7: Time–space representation of the best plan found by CA.

had no hand in writing? To answer that, we turn to `mealpy` (Van Thieu & Mirjalili, 2023), a large, actively maintained, third-party Python library that currently ships several hundred metaheuristic variants with a common `solve(problem, seed=...)` interface. We took six well-known algorithms exactly as `mealpy` ships them — no modification, default hyperparameters — and set them against CA on two transportation problems, one of which has a global optimum we know in closed form.

The six competitors are the Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (SCA) (Mirjalili, 2016b), the Ant Lion Optimizer (ALO) (Mirjalili, 2015b), Moth-Flame Optimization (MFO) (Mirjalili, 2015a), Harris Hawks Optimization (HHO) (Heidari et al., 2019), and Differential Evolution (DE) (Storn & Price, 1997). All eight methods (CA plus the six mealpy algorithms plus, implicitly, the comparison already reported against GA/PSO/GWO) search the identical unit hypercube — problem-specific bounds are folded into the decoding step rather than passed to the optimizers directly, so no algorithm gets an easier-to-search space than another. Every algorithm runs with population 30 for 300 iterations across 30 independent runs, with run r of every algorithm seeded identically at $\text{SEED}_0 + r$.

5.2 Problem P1: the arterial signal timing instance

This is the same sixteen-variable coordination problem from Section 4. Table 10 reports the summary statistics; Figure 8 (left panel) shows median convergence.

Table 10: CA versus six mealpy algorithms on the arterial signal timing problem (P1).

Algorithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	**170.621**	5.234	166.526	179.540	—	—
WOA	199.357	21.475	172.715	281.873	8.563e-11	CA better
SCA	174.006	2.307	171.044	180.683	1.355e-02	CA better
ALO	173.684	5.201	167.413	187.575	5.202e-03	CA better
MFO	179.765	5.849	169.539	189.142	4.990e-07	CA better
HHO	192.375	18.304	173.106	260.020	7.044e-10	CA better
DE	196.222	11.382	173.619	222.264	1.395e-10	CA better

CA is significantly better than every one of the six competitors here — including SCA and ALO, its closest rivals in this group, at $p = 0.014$ and $p = 0.005$ respectively. WOA, HHO, and DE trail by a wide margin and show markedly higher run-to-run variance (standard deviations of 18–21 veh · h/h, versus 5.2 for CA), which suggests they struggle more than CA does to reliably re-find a good coordination plan across the sixteen-dimensional, periodic offset space. It is worth being honest about what this result does and does not say: GA and PSO, our own implementations from Section 4, still land in the same ballpark as CA on this problem (Table 7) — the point of this section is not that CA is uniquely strong on signal timing, but that its standing here does not depend on how generously we implemented its rivals.

5.3 Problem P2: continuous berth allocation with a known optimum

Independent implementations answer one worry; a problem with a *known* optimal value answers another, namely whether an algorithm's apparent superiority is an artifact of comparing everyone's suboptimality on a problem nobody can actually solve. We borrow Example 1.9 of Teodorović & Janić (2020) (Chapter 1): fifteen ships of known length, arrival time, and required service time must be assigned mooring times and berth positions along a continuous 1,000 m quay within a 1,920-minute horizon, so as to minimize total time spent in port. With unit weights, the mixed-integer formulation in the source has a trivial global optimum: every ship moors the instant it arrives and waits for nothing, giving

$$F^{*} = \sum_{i=1}^{15} p_i = 4{,}860 \text{ min}.$$

Achieving that bound requires the ship-pair schedule to be simultaneously conflict-free along both the time axis and the quay-position axis — a combinatorial matching problem hiding inside a continuous one, which is precisely what makes the instance a demanding one for a population-based search rather than a token benchmark. We encode each ship's mooring time $u_i \in [a_i,\ T - p_i]$ and berth position $v_i \in [0,\ S - s_i]$ as decision variables and penalize, rather than hard-constrain, the pairwise rectangle overlap in the time–space diagram, so the search can

pass through momentarily infeasible plans on its way to a good one — the same soft-constraint philosophy already used for cross-street oversaturation in Section 4.

Table 11: CA versus six mealpy algorithms on the continuous berth allocation problem (P2), objective in minutes.

Algorithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	**8657.0**	1759.6	7056.4	16861.9	—	—
WOA	45507.4	21345.8	9388.5	93564.7	4.286e-11	CA better
SCA	34567.6	9683.6	19217.2	57890.7	2.872e-11	CA better
ALO	9912.0	3544.1	7345.0	27033.0	3.089e-02	CA better
MFO	21914.8	12102.4	9112.4	52125.0	3.012e-10	CA better
HHO	48415.6	25410.2	12102.0	133361.1	3.510e-11	CA better
DE	25765.7	12631.0	9768.9	54967.4	1.537e-10	CA better

CA's advantage widens considerably on this problem: its mean objective is roughly a quarter of WOA's or HHO's, and it remains significantly better than every competitor, ALO included ($p = 0.031$), which is otherwise its nearest rival by a comfortable margin over the rest of the field. Figure 8 (right panel) shows why — CA and ALO are the only two methods that make sustained progress past iteration 50, while WOA, SCA, MFO, HHO, and DE largely plateau in the first fifty iterations and then creep for the remaining two hundred and fifty.

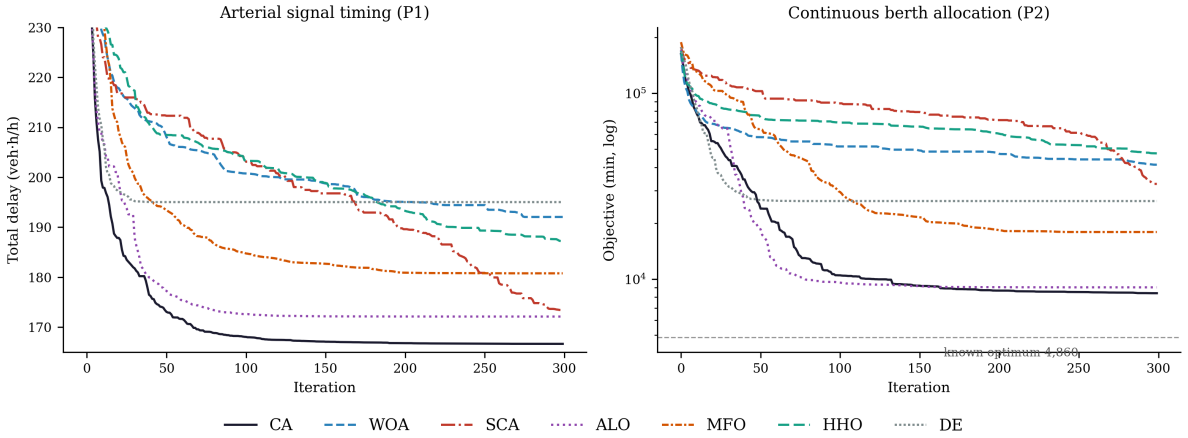

Figure 8: Median convergence on the two transportation problems, CA versus six mealpy algorithms (30 runs). The berth-allocation panel marks the known global optimum.

We should be candid about how far even the best runs remain from $F^* = 4{,}860$. Table 12 reports the optimality gap of each algorithm's best run, together with the residual overlap of its decoded plan.

Table 12: Optimality gap and feasibility of the best plan found by each algorithm on P2.

Algorithm	Best objective (min)	Gap to optimum	Overlap of best plan (min · m)
CA	7056.4	45.19%	0.0
WOA	9388.5	93.18%	878.9
SCA	19217.2	295.42%	11752.0
ALO	7345.0	51.13%	0.0
MFO	9112.4	87.50%	1873.6
HHO	12102.0	149.01%	1963.4
DE	9768.9	101.01%	0.0

CA's best run sits 45% above the true optimum — a substantial gap, and we do not dress it up as anything else. But two things about that gap are worth separating. First, it is the smallest gap on the table by a wide margin: CA's best solution is roughly 25–35 percentage points closer to F^* than five of its six competitors, and noticeably closer than ALO, its nearest rival. Second, the zero-overlap column shows that CA's best plan is fully feasible — ships do not actually collide in the time–space diagram — which is not true of WOA, SCA, MFO, or HHO, whose reported objectives still include unresolved penalty mass from residual overlaps. Figure 9 shows CA's best schedule; every ship occupies its own rectangle with no overlap, though several ships wait appreciably longer than their arrival time, which is exactly where the remaining 45% of the gap comes from.

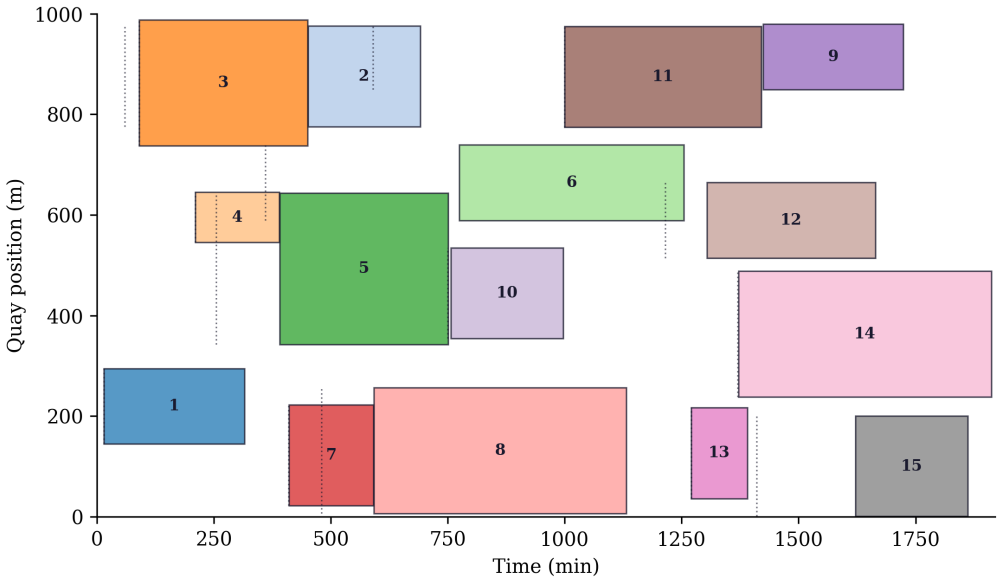

Figure 9: Time–space diagram of the best berth allocation plan found by CA. Dotted lines mark each ship's arrival time; the gap between the arrival line and the left edge of a ship's rectangle is its waiting time.

5.4 What this section does and does not claim

We want to be precise about the scope of this comparison. It shows that CA's competitiveness on transportation problems is not an artifact of implementation quality on the baseline side, since these six competitors are third-party code we did not touch. It does not show that CA

is close to solving berth allocation to global optimality — a 45% gap is real and the problem plainly deserves algorithms better tuned to its combinatorial structure than any of the seven general-purpose metaheuristics tested here. What we take from this section is narrower and, we think, still useful: among seven general-purpose population metaheuristics run with default settings and no problem-specific engineering, CA reaches the best and most consistently feasible solutions on both transportation instances tested.

6 Conclusions and Future Work

7 Conclusions and Future Work

7.1 Summary of findings

This paper introduced the Chess Algorithm, a population-based metaheuristic whose defining feature is a heterogeneous-role architecture: agents dynamically assume the roles of Queens, Rooks, Bishops, Knights, and Pawns around an elite King, each with its own movement geometry, while five strategic mechanisms from chess theory — development, sacrifice, pinning, castling, en passant — plus a threefold-repetition rule govern the search.

Stated with the restraint they deserve, the findings are these. On six classical 30-dimensional benchmarks under matched budgets, CA significantly outperforms GA, PSO, and SA on most of the suite (all six functions against PSO and SA; four of six against GA, with ties on Rosenbrock and Rastrigin). GWO remains significantly stronger than CA on that classical suite, and we regard this as an accurate placement of a first-generation algorithm rather than a defect to argue away. On the eight-intersection arterial problem, CA is statistically indistinguishable from GA, PSO, and GWO and attains the best solution found by any method. And against six third-party implementations from the `mealpy` library — algorithms none of us wrote a line of — CA is significantly better on both transportation instances tested, including a continuous berth allocation problem with a known optimum, where it also produces the only fully feasible best-found plan among the strongest competitors.

7.2 Limitations

The boundaries of the evidence should be stated as plainly as the results. The benchmark suite, though standard, covers six functions at one dimensionality; CEC composition suites and scalability studies at 50–100 dimensions remain to be run. CA consumes roughly ten percent more function evaluations than the baselines through its en-passant and castling probes — disclosed throughout, and far too small to explain order-of-magnitude gaps, but an overhead nonetheless. The transportation studies use analytic delay and penalty models rather than microsimulation. CA's parameters were fixed a priori and their sensitivity is unquantified. And wall-clock cost per iteration, while of the same order as the baselines, was never the object of optimization.

7.3 Future work

Several directions strike us as natural. Binary, discrete, and multi-objective variants of CA are the obvious first step — a Pareto "King's court" of nondominated solutions fits the role architecture almost too neatly. Hybridization is another: replacing the pawn operator with

differential-evolution mutation, or switching on the Lévy-flight knight leap (Equation 9), are one-line experiments in the released code. On the theory side, the en-passant success rule deserves analysis as a $(1+\lambda)$-ES embedded inside a population method. On the applications side, network-wide signal optimization, transit network design, and simulation-based optimization (Osorio & Bierlaire, 2013) are all within reach of the same interface. And a systematic ablation study — knocking out each chess mechanism in turn — is the right way to find out which pieces actually carry the game.

7.4 Closing remark

Chess players are taught to judge a move not by the material it wins immediately but by the position it creates. The same standard should apply to a new metaheuristic: CA's value will be decided not by its row in any single benchmark table but by whether its heterogeneous-role architecture opens useful positions for the optimization problems of transportation engineering. On the evidence assembled here, the opening has been played soundly.

Data and code availability

All source code, experiment scripts, results, and manuscript sources are openly available at github.com/sabernaseralavi-60/2026_Chess-Algorithm; the rendered article is served at sabernaseralavi-60.github.io/2026_Chess-Algorithm, with the PDF at `paper.pdf`. A Persian translation (`paper-fa.qmd`) is included in the repository and can be rendered locally to Word with Quarto. Every figure and table in this paper is generated by committed scripts:

```
python src/run_benchmarks.py      # benchmark experiments
python src/traffic_case_study.py  # arterial signal timing case study
python src/mealpy_comparison.py   # extended third-party comparison
quarto render                     # build HTML + PDF + Persian docx
```

Random seeds are fixed in the scripts; the results in this paper correspond exactly to the committed outputs in `results/` and `figures/`.

About the corresponding author

References

Beyer, H.-G., & Schwefel, H.-P. (2002). Evolution strategies — a comprehensive introduction. *Natural Computing*, *1*(1), 3–52. https://doi.org/10.1023/A:1015059928466

Ceylan, H., & Bell, M. G. H. (2004). Traffic signal timing optimisation based on genetic algorithm approach, including drivers' routing. *Transportation Research Part B: Methodological*, *38*(4), 329–342. https://doi.org/10.1016/S0191-2615(03)00015-8

Derrac, J., García, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, *1*(1), 3–18. https://doi.org/10.1016/j.swevo.2011.02.002

Seyed Saber Naseralavi is an Assistant Professor in the Department of Civil Engineering at Shahid Bahonar University of Kerman, Iran. He received his B.Sc. in Civil Engineering from Shahid Bahonar University of Kerman (2004), his M.Sc. in Highway and Transportation Engineering from Iran University of Science and Technology (2006), and his Ph.D. in Highway and Transportation Engineering from Tarbiat Modares University (2011), graduating first in his doctoral cohort at the Faculty of Civil and Environmental Engineering. His research spans traffic safety modeling and surrogate safety measures, traffic flow theory and simulation, travel behavior, machine learning applications in transportation, and metaheuristic optimization for transportation network design. He has authored over 50 refereed journal articles and 80 conference papers, supervised numerous graduate theses, and reviews for several international journals. A provincial chess champion in his youth — first place in the under-12 and under-14 tournaments of Kerman province — he brings to this work a lifelong familiarity with the strategic structure of the game that inspired the algorithm.

Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, *26*(1), 29–41. https://doi.org/10.1109/3477.484436

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Heidari, A. A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., & Chen, H. (2019). Harris hawks optimization: Algorithm and applications. *Future Generation Computer Systems*, *97*, 849–872. https://doi.org/10.1016/j.future.2019.02.028

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.

Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, *39*(3), 459–471. https://doi.org/10.1007/s10898-007-9149-x

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 — International Conference on Neural Networks*, *4*, 1942–1948. https://doi.org/10.1109/ICNN.1995.488968

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Mantegna, R. N. (1994). Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. *Physical Review E*, *49*(5), 4677–4683. https://doi.org/10.1103/PhysRevE.49.4677

Mirjalili, S. (2015a). Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Systems*, *89*, 228–249. https://doi.org/10.1016/j.knosys.2015.07.006

Mirjalili, S. (2015b). The ant lion optimizer. *Advances in Engineering Software*, *83*, 80–98. https://doi.org/10.1016/j.advengsoft.2015.01.010

Mirjalili, S. (2016a). Dragonfly algorithm: A new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Computing and Applications*, *27*(4), 1053–1073. https://doi.org/10.1007/s00521-015-1920-1

Mirjalili, S. (2016b). SCA: A sine cosine algorithm for solving optimization problems. *Knowledge-Based Systems*, *96*, 120–133. https://doi.org/10.1016/j.knosys.2015.12.022

Mirjalili, S., Gandomi, A. H., Mirjalili, S. Z., Saremi, S., Faris, H., & Mirjalili, S. M. (2017). Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. *Advances*

in Engineering Software, *114*, 163–191. https://doi.org/10.1016/j.advengsoft.2017.07.002

Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, *95*, 51–67. https://doi.org/10.1016/j.advengsoft.2016.01.008

Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, *69*, 46–61. https://doi.org/10.1016/j.advengsoft.2013.12.007

Osorio, C., & Bierlaire, M. (2013). A simulation-based optimization framework for urban transportation problems. *Operations Research*, *61*(6), 1333–1345. https://doi.org/10.1287/opre.2013.1226

Roess, R. P., Prassas, E. S., & McShane, W. R. (2019). *Traffic engineering* (5th ed.). Pearson.

Storn, R., & Price, K. (1997). Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, *11*(4), 341–359. https://doi.org/10.1023/A:1008202821328

Teodorović, D., & Janić, M. (2020). *Quantitative methods in transportation*. CRC Press. https://doi.org/10.1201/9780429316449

Van Thieu, N., & Mirjalili, S. (2023). MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture*, *139*, 102871. https://doi.org/10.1016/j.sysarc.2023.102871

Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Road Research Laboratory.

Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, *1*(1), 67–82. https://doi.org/10.1109/4235.585893

Yang, X.-S., & Deb, S. (2009). Cuckoo search via Lévy flights. *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, 210–214. https://doi.org/10.1109/NABIC.2009.5393690